

Installation Guide: CBC Solver for Pyomo in VS Code

October 2026

Introduction

This document explains how to install the CBC (Coin-or branch and cut) solver for use with Pyomo in Visual Studio Code on a 64-bit Windows computer. The installation uses the `cbcbox` Python package, which provides a precompiled Windows binary and bundles all necessary dependencies (OpenBLAS, libgfortran, etc.), avoiding manual PATH conflicts and DLL errors.

After completing this guide, you will be able to create a Pyomo `SolverFactory('cbc')` object and verify that the solver is available.

Prerequisites

Ensure the following are already installed and verified:

- Visual Studio Code (with Python extension)
- Python 3.11 or later (added to PATH)
- Pyomo (`pip install pyomo`)

Step 1: Install the cbcbox Package

Open the VS Code terminal (PowerShell) and run:

```
pip install cbcbox
```

This downloads and installs the `cbcbox` package, which contains the optimized CBC executable and all dependencies.

Expected output (truncated):

```
Collecting cbcbox
  Downloading cbcbox-2.929-cp313-cp313-win_amd64.whl (72.2 MB)
Installing collected packages: cbcbox
Successfully installed cbcbox-2.929
```

Note: The download is approximately 72 MB because the wheel ships both a Haswell-optimised (AVX2) binary and a generic binary for maximum CPU compatibility.

Step 2: Locate the CBC Executable

The `cbcbox` package places the CBC binary inside a subdirectory of the package installation folder. The exact subdirectory name (`cbc_dist` or `cbc_dist_avx2`) depends on the CPU architecture and the build variant selected automatically at import time.

Directory Structure

The relevant directory structure inside the cbcbox package is:

```
cbcbox/  
|-- cbc_dist/ <-- generic build (all platforms)  
| |-- bin/  
| | |-- cbc.exe <-- CBC MIP solver (Windows)  
| | '-- clp.exe <-- Clp LP solver  
| '-- lib/  
| '-- libCbc.dll <-- CBC shared library  
'-- cbc_dist_avx2/ <-- AVX2-optimised build (x86_64)  
    '-- bin/  
        '-- cbc.exe
```

Important: The executable name on Windows is `cbc.exe`. Do not hard-code the subdirectory name (`cbc_dist` or `cbc_dist_avx2`), because the package may switch between them on different systems or after updates.

Dynamic Location Script

Because the subdirectory name can vary, use a Python script that dynamically searches for `cbc.exe` inside the cbcbox package. Create a new file named `test_cbc.py` in VS Code and paste the following code:

```
import os  
import cbcbox  
from pyomo.environ import SolverFactory  
  
# Get the base directory of the cbcbox package  
cbcbox_dir = os.path.dirname(cbcbox.__file__)  
cbc_executable = None  
  
# Search for cbc.exe inside the cbcbox package directory  
for root, dirs, files in os.walk(cbcbox_dir):  
    for file in files:  
        if file.lower() == 'cbc.exe':  
            cbc_executable = os.path.join(root, file)  
            break  
    if cbc_executable:  
        break  
  
if cbc_executable:  
    print(f"Found CBC executable at: {cbc_executable}")  
    # Create the solver with the explicit executable path  
    solver = SolverFactory('cbc', executable=cbc_executable)  
    # Test if the solver is available  
    print("CBC available:", solver.available())  
else:  
    print("Could not find cbc.exe inside the cbcbox package. "  
          "You may need to install it manually (see Section 9).")
```

Verified Output on the Target System

When the script above was executed on the target system, it produced the following output:

```
Found CBC executable at: C:\Users\mngpt\AppData\Local\Programs\Python\Python313\Lib\site-  
packages\cbcbox\cbc_dist\bin\cbc.exe  
CBC available: True
```

Cross-check: The official cbcbox documentation confirms that the CBC binary path follows the pattern `.../cbcbox/cbc_dist/bin/cbc` (or `cbc_dist_avx2/bin/cbc` on AVX2-capable CPUs). The dynamic search in the script correctly handles both variants.

Step 3: Configure Pyomo to Use CBC

We have already configured Pyomo by passing the `executable` argument to `SolverFactory`. This approach bypasses the system PATH entirely, which is the most reliable method on Windows because it avoids conflicts with other CBC installations that may be present (e.g., from PuLP or a standalone AMPL download).

For repeated use in multiple scripts, you can define a reusable function:

```
import os
import cbcbox
from pyomo.environ import SolverFactory

def get_cbc_solver():
    """Locate and return a Pyomo CBC solver instance."""
    cbcbox_dir = os.path.dirname(cbcbox.__file__)
    for root, dirs, files in os.walk(cbcbox_dir):
        for file in files:
            if file.lower() == 'cbc.exe':
                exe_path = os.path.join(root, file)
                return SolverFactory('cbc', executable=exe_path)
    raise FileNotFoundError("cbc.exe not found inside the cbcbox package.")

# Usage
solver = get_cbc_solver()
print("CBC available:", solver.available())
```

Step 4: Verify the Installation

Run the verification script from the VS Code terminal:

```
python test_cbc.py
```

Expected output:

```
Found CBC executable at: C:\Users\...\site-packages\cbcbox\cbc_dist\bin\cbc.exe
CBC available: True
```

If you see `CBC available: True`, the installation is successful and the solver is ready for use with Pyomo models.

Important Notes

Performance Tips

- **Use multiple threads:** CBC built with `-enable-cbc-parallel` supports multi-threaded branch-and-cut. Pass `-threads=N` to the solver options to distribute the search tree across N CPU cores.

```
solver = SolverFactory('cbc', executable=cbc_executable)
solver.options['threads'] = 8 # adjust to your CPU core count
```

- **Tighten Big-M constraints:** In your Pyomo model, replace weak Big-M values (e.g., 100000000) with the tightest valid bound (e.g., `min(supply[i], demand[k])`). This significantly reduces the branch-and-bound tree size and solve time.

- Set time limits and gaps for large models:

```
solver.options['sec'] = 60 # stop after 60 seconds
solver.options['ratio'] = 0.05 # stop when gap <= 5%
```

Known Warning: “Unable to open file .\1”

When Pyomo calls CBC, you may see the following message in the solver log:

```
Unable to open file .\1
```

This is a **harmless, known bug** in the Pyomo–CBC interface on Windows. CBC attempts to read a parameter file named 1 and fails silently. It does **not** affect the solver’s operation or the solution quality. You can safely ignore this message.

Updating the cbcbox Package

If you update the package in the future:

```
pip install --upgrade cbcbox
```

The dynamic search script (Section 4.2) will automatically locate the new executable path without requiring any changes to your code.

Coexistence with Other Solvers

The cbcbox package does not conflict with Gurobi, SCIP, or other solvers. Each solver is managed independently by Pyomo. However, if you have another CBC installation (e.g., from PuLP or a standalone AMPL download) on your system PATH, Pyomo’s `SolverFactory('cbc')` without an explicit `executable` argument may find the wrong binary. Using the explicit `executable` path as shown in this guide avoids this ambiguity.

Alternative: Manual Installation

If you prefer not to use the cbcbox package, you can install CBC manually. This method is not recommended for most users because it requires manual PATH configuration and may result in missing DLL errors.

1. Download the CBC binary from the AMPL open-source solvers page:

<https://ampl.com/dl/open/cbc/cbc-win64.zip>

2. Extract to a dedicated folder:

```
New-Item -ItemType Directory -Force -Path "C:\solvers\cbc"
Expand-Archive -Path "$env:USERPROFILE\Downloads\cbc-win64.zip" -DestinationPath "C:\solvers\cbc" -Force
```

3. Add the folder to your User PATH:

```
$cbcPath = "C:\solvers\cbc"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User") + ";" + $cbcPath, "User")
```

4. Restart the VS Code terminal (close and reopen).

5. Verify the PATH:

```
where.exe cbc
```

Expected: C:\solvers\cbc\cbc.exe

6. Use in Pyomo without the executable argument:

```
from pyomo.environ import SolverFactory
solver = SolverFactory('cbc')
print("CBC available:", solver.available())
```

Installation of HIGHS Solver

Step 1: Install the highspy Package

Open the VS Code terminal (PowerShell) and run:

```
pip install highspy
```